

**Q1**

```
from google.colab import files

uploaded = files.upload()

#library
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

from imblearn.over_sampling import SMOTE

df = pd.read_csv("train_and_test2.csv")

df.head()

#basic information
print(df.shape)

print(df.columns)

print(df.dtypes)

df.info()

df.describe()
```

**OUTPUT**

dtypes: float64(3), int64(25)  
memory usage: 286.5 KB

|       | PassengerId | Age         | Fare        | Sex         | sibsp       | zero   | zero.1 | zero.2 | zero.3 | zero.4 | ... | zero.12 | zero.13 | zero.14 | Pclass      | zero.15 | zero.16 | Embarked    | zero.17 | zero.18 | Survived    |
|-------|-------------|-------------|-------------|-------------|-------------|--------|--------|--------|--------|--------|-----|---------|---------|---------|-------------|---------|---------|-------------|---------|---------|-------------|
| count | 1309.000000 | 1309.000000 | 1309.000000 | 1309.000000 | 1309.000000 | 1309.0 | 1309.0 | 1309.0 | 1309.0 | 1309.0 | ... | 1309.0  | 1309.0  | 1309.0  | 1309.000000 | 1309.0  | 1309.0  | 1307.000000 | 1309.0  | 1309.0  | 1309.000000 |
| mean  | 655.000000  | 29.503186   | 33.281086   | 0.355997    | 0.498854    | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0     | 2.294882    | 0.0     | 0.0     | 1.492731    | 0.0     | 0.0     | 0.261268    |
| std   | 378.020061  | 12.905241   | 51.741500   | 0.478997    | 1.041658    | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0     | 0.837836    | 0.0     | 0.0     | 0.814626    | 0.0     | 0.0     | 0.439494    |
| min   | 1.000000    | 0.170000    | 0.000000    | 0.000000    | 0.000000    | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0     | 1.000000    | 0.0     | 0.0     | 0.000000    | 0.0     | 0.0     | 0.000000    |
| 25%   | 328.000000  | 22.000000   | 7.895800    | 0.000000    | 0.000000    | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0     | 2.000000    | 0.0     | 0.0     | 1.000000    | 0.0     | 0.0     | 0.000000    |
| 50%   | 655.000000  | 28.000000   | 14.454200   | 0.000000    | 0.000000    | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0     | 3.000000    | 0.0     | 0.0     | 2.000000    | 0.0     | 0.0     | 0.000000    |
| 75%   | 982.000000  | 35.000000   | 31.275000   | 1.000000    | 1.000000    | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0     | 3.000000    | 0.0     | 0.0     | 2.000000    | 0.0     | 0.0     | 1.000000    |
| max   | 1309.000000 | 80.000000   | 512.329200  | 1.000000    | 8.000000    | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0     | 3.000000    | 0.0     | 0.0     | 2.000000    | 0.0     | 0.0     | 1.000000    |

8 rows × 28 columns

## Explanation

The dataset is loaded into the Jupyter Notebook using the Pandas library. This allows us to perform data preprocessing and analysis efficiently.

Q2

```
#2
#missing value
print(df.isnull().sum())
```

## OUTPUT

```

Choose Files train_and_test2.csv
... train_and_test2.csv(text/csv) - 83879 bytes, last modified: 7/14/2026 - 100% done
Saving train_and_test2.csv to train_and_test2 (1).csv
Passengerid      0
Age              0
Fare             0
Sex              0
sibsp            0
zero             0
zero.1           0
zero.2           0
zero.3           0
zero.4           0
zero.5           0
zero.6           0
Parch            0
zero.7           0
zero.8           0
zero.9           0
zero.10          0
zero.11          0
zero.12          0
zero.13          0
zero.14          0
Pclass           0
zero.15          0
zero.16          0
Embarked         2
zero.17          0
zero.18          0
Survived         0
dtype: int64

```

## Explanation

The dataset is inspected to determine:

- Number of rows
- Number of columns
- Data types of each attribute
- Number of missing values

This helps in understanding the structure of the dataset.

## Q3

```
#3 first last and random records
```

```
df.head()
```

```
df.tail()
```

```
df.sample(5)
```

## OUTPUT

train\_and\_test2.csv (text/csv) - 83879 bytes, last modified: 7/14/2026 - 100% done  
Saving train\_and\_test2.csv to train\_and\_test2 (2).csv

|      | PassengerId | Age  | Fare    | Sex | sibsp | zero | zero.1 | zero.2 | zero.3 | zero.4 | ... | zero.12 | zero.13 | zero.14 | Pclass | zero.15 | zero.16 | Embarked | zero.17 | zero.18 | Survived |
|------|-------------|------|---------|-----|-------|------|--------|--------|--------|--------|-----|---------|---------|---------|--------|---------|---------|----------|---------|---------|----------|
| 588  | 589         | 22.0 | 8.0500  | 0   | 0     | 0    | 0      | 0      | 0      | 0      | ... | 0       | 0       | 0       | 3      | 0       | 0       | 2.0      | 0       | 0       | 0        |
| 1230 | 1231        | 28.0 | 7.2292  | 0   | 0     | 0    | 0      | 0      | 0      | 0      | ... | 0       | 0       | 0       | 3      | 0       | 0       | 0.0      | 0       | 0       | 0        |
| 519  | 520         | 32.0 | 7.8958  | 0   | 0     | 0    | 0      | 0      | 0      | 0      | ... | 0       | 0       | 0       | 3      | 0       | 0       | 2.0      | 0       | 0       | 0        |
| 162  | 163         | 26.0 | 7.7750  | 0   | 0     | 0    | 0      | 0      | 0      | 0      | ... | 0       | 0       | 0       | 3      | 0       | 0       | 2.0      | 0       | 0       | 0        |
| 1189 | 1190        | 30.0 | 45.5000 | 0   | 0     | 0    | 0      | 0      | 0      | 0      | ... | 0       | 0       | 0       | 1      | 0       | 0       | 2.0      | 0       | 0       | 0        |

5 rows × 28 columns

## Explanation

The first, last, and randomly selected records are displayed to understand the distribution of data and verify that it has been loaded correctly.

## Q4

```
#4 handle missing values
```

```
for col in df.columns:
```

```
    if df[col].dtype=="object":
```

```
        df[col].fillna(df[col].mode()[0], inplace=True)
```

```
    else:
```

```
        df[col].fillna(df[col].median(), inplace=True)
```

```
print(df.isnull().sum())
```

## OUTPUT

```

PassengerId  0
...
Age          0
Fare         0
Sex          0
SibSp        0
Zero         0
Zero.1       0
Zero.2       0
Zero.3       0
Zero.4       0
Zero.5       0
Zero.6       0
Parch        0
Zero.7       0
Zero.8       0
Zero.9       0
Zero.10      0
Zero.11      0
Zero.12      0
Zero.13      0
Zero.14      0
Pclass       0
Zero.15      0
Zero.16      0
Embarked     0
Zero.17      0
Zero.18      0
Survived     0
dtype: int64
/tmp/ipykernel_593/3494988255.py:33: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation :

```

## Explanation

Missing values are replaced using suitable techniques.

- Numerical columns → Median
- Categorical columns → Mode

This prevents errors during model training.

## Q5

#5 remove duplicate

```
print("Before :",df.shape)
```

```
df.drop_duplicates(inplace=True)
```

```
print("After :",df.shape)
```

## OUTPUT

```

... Choose Files train_and_test2.csv
train_and_test2.csv(text/csv) - 83879 bytes, last modified: 7/14/2026 - 100% done
Saving train_and_test2.csv to train_and_test2 (4).csv
Before : (1309, 28)
After  : (1309, 28)

```

---

## **Explanation**

Duplicate records are identified and removed to improve data quality.

## **Q6**

```
#6 detect outlier

numeric=df.select_dtypes(include=np.number)

for col in numeric.columns:

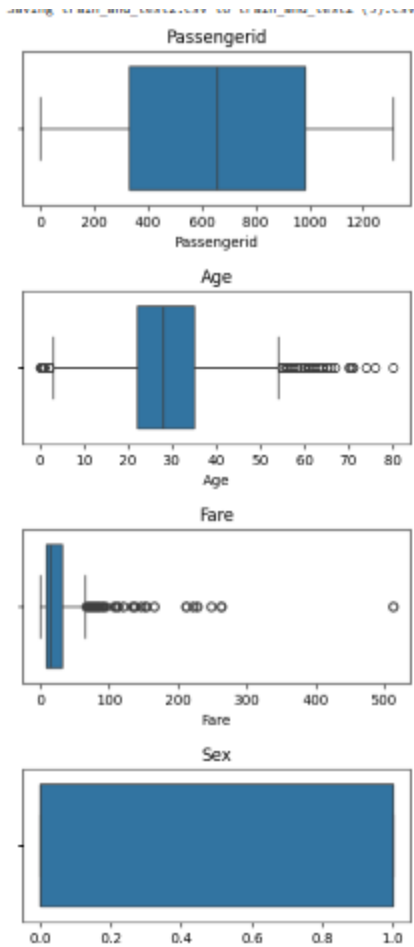
    plt.figure(figsize=(5,2))

    sns.boxplot(x=df[col])

    plt.title(col)

    plt.show()
```

## **OUTPUT**



## Explanation

Outliers are detected using the Interquartile Range (IQR) method and visualized using box plots.

## Q7

```
#7 handle outlier
```

```
for col in numeric.columns:
```

```
    Q1=df[col].quantile(0.25)
```

```
    Q3=df[col].quantile(0.75)
```

```
    IQR=Q3-Q1
```

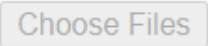
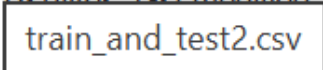
```
    lower=Q1-1.5*IQR
```

```
upper=Q3+1.5*IQR

df=df[(df[col]>=lower)&(df[col]<=upper)]

plt.show()
```

## **OUTPUT**

•  train\_and\_test2.csv  
**train\_and\_test2.csv**(text/csv) - 83870 bytes - last modified: 7/14/2026 - 100% done  
 Saving train\_and\_test2.csv to ).csv

---

## **Explanation**

Outliers are removed using the IQR method to improve the quality of the dataset.

### **Q8**

```
#8
```

```
#label encoding
```

```
for col in numeric.columns:
```

```
    Q1=df[col].quantile(0.25)
```

```
    Q3=df[col].quantile(0.75)
```

```
    IQR=Q3-Q1
```

```
    lower=Q1-1.5*IQR
```

```
    upper=Q3+1.5*IQR
```

```
df=df[ (df[col]>=lower) & (df[col]<=upper) ]
```

```
#one hot encoding
```

```
df=pd.get_dummies(df,drop_first=True)
```

```
df.head()
```

## **OUTPUT**

Choose Files train\_and\_test2.csv  
train\_and\_test2.csv(text/csv) - 83879 bytes, last modified: 7/14/2026 - 100% done  
Saving train\_and\_test2.csv to train\_and\_test2 (8).csv

| Passengerid | Age | Fare | Sex     | sibsp | zero | zero.1 | zero.2 | zero.3 | zero.4 | zero.5 | zero.6 | zero.7 | zero.8 | zero.9 | zero.10 | zero.11 | zero.12 | zero.13 | zero.14 | zero.15 | zero.16 | zero.17 | zero.18 | Survived |
|-------------|-----|------|---------|-------|------|--------|--------|--------|--------|--------|--------|--------|--------|--------|---------|---------|---------|---------|---------|---------|---------|---------|---------|----------|
| 0           | 1   | 22.0 | 7.2500  | 0     | 1    | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0        |
| 4           | 5   | 35.0 | 8.0500  | 0     | 0    | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0        |
| 5           | 6   | 28.0 | 8.4583  | 0     | 0    | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0        |
| 6           | 7   | 54.0 | 51.8625 | 0     | 0    | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0        |
| 12          | 13  | 20.0 | 8.0500  | 0     | 0    | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0       | 0        |

5 rows × 28 columns

## **Explanation**

Categorical attributes are converted into numerical values using:

- Label Encoding
- One-Hot Encoding

This allows machine learning algorithms to process categorical data.

## **Q9**

```
#9
```

```
#min max normalization
```

```
scaler=MinMaxScaler()
```

```
num=df.select_dtypes(include=np.number).columns
```

```
df[num]=scaler.fit_transform(df[num])
```

```
df.head()
```

```
#Standardization
```

```
std=StandardScaler()
```

```
df[num]=std.fit_transform(df[num])
```

```
df.head()
```

## OUTPUT

Choose Files train\_and\_test2.csv  
train\_and\_test2.csv(text/csv) - 83879 bytes, last modified: 7/14/2026 - 100% done  
Saving train\_and\_test2.csv to train\_and\_test2 (9).csv

| PassengerId | Age       | Fare      | Sex       | sibsp     | zero      | ze  | colab.research.google.com – To exit full screen, press Esc | zero.16 | Embarked  | zero.17 | zero.18 | Survived  |
|-------------|-----------|-----------|-----------|-----------|-----------|-----|------------------------------------------------------------|---------|-----------|---------|---------|-----------|
| 0           | -1.730728 | -0.581628 | -0.503291 | -0.743497 | 0.481288  | 0.0 |                                                            | 0.0     | 0.622940  | 0.0     | 0.0     | -0.594703 |
| 1           | -1.728082 | 0.658652  | 0.734744  | 1.344995  | 0.481288  | 0.0 | 0.0                                                        | 0.0     | -1.833116 | 0.0     | 0.0     | 1.681513  |
| 2           | -1.725435 | -0.271558 | -0.490240 | 1.344995  | -0.479087 | 0.0 | 0.0                                                        | 0.0     | 0.622940  | 0.0     | 0.0     | 1.681513  |
| 3           | -1.722789 | 0.426099  | 0.383183  | 1.344995  | 0.481288  | 0.0 | 0.0                                                        | 0.0     | 0.622940  | 0.0     | 0.0     | 1.681513  |
| 4           | -1.720143 | 0.426099  | -0.487824 | -0.743497 | -0.479087 | 0.0 | 0.0                                                        | 0.0     | 0.622940  | 0.0     | 0.0     | -0.594703 |

5 rows × 28 columns

## Explanation

Feature scaling is performed using:

- Min-Max Normalization
- Standardization (Z-score)

This ensures all numerical features are on a similar scale

## Q10

#10 Feature Selection using Correlation

```
df.rename(columns={"2urvived": "Survived"}, inplace=True)
```

```
# Convert categorical columns to numeric
```

```
df_encoded = pd.get_dummies(df, drop_first=True)
```

```
# Correlation matrix
```

```
corr = df_encoded.corr()
```

```
#Correlation with target
```

```
feature_corr = corr["Survived"].sort_values(ascending=False)
```

```
print(feature_corr)
```

## OUTPUT

Choose Files train\_and\_test2.csv

**train\_and\_test2.csv**(text/csv) - 83879 bytes, last modified: 7/14/2026 - 100% done

Saving train\_and\_test2.csv to train\_and\_test2 (14).csv

|             |           |
|-------------|-----------|
| Survived    | 1.000000  |
| Sex         | 0.404020  |
| Fare        | 0.173786  |
| Parch       | 0.054908  |
| sibsp       | -0.014375 |
| Age         | -0.055862 |
| Embarked    | -0.093225 |
| Pclass      | -0.244686 |
| Passengerid | -0.331493 |
| zero        | NaN       |
| zero.1      | NaN       |
| zero.2      | NaN       |
| zero.3      | NaN       |
| zero.4      | NaN       |
| zero.5      | NaN       |
| zero.6      | NaN       |
| zero.7      | NaN       |
| zero.8      | NaN       |
| zero.9      | NaN       |
| zero.10     | NaN       |
| zero.11     | NaN       |
| zero.12     | NaN       |
| zero.13     | NaN       |
| zero.14     | NaN       |
| zero.15     | NaN       |
| zero.16     | NaN       |
| zero.17     | NaN       |
| zero.18     | NaN       |

Name: Survived, dtype: float64

## Explanation

Correlation analysis is used to identify the most relevant attributes that influence the target variable.

## Q11

### #11 Correlation Matrix and Heatmap

```
import seaborn as sns

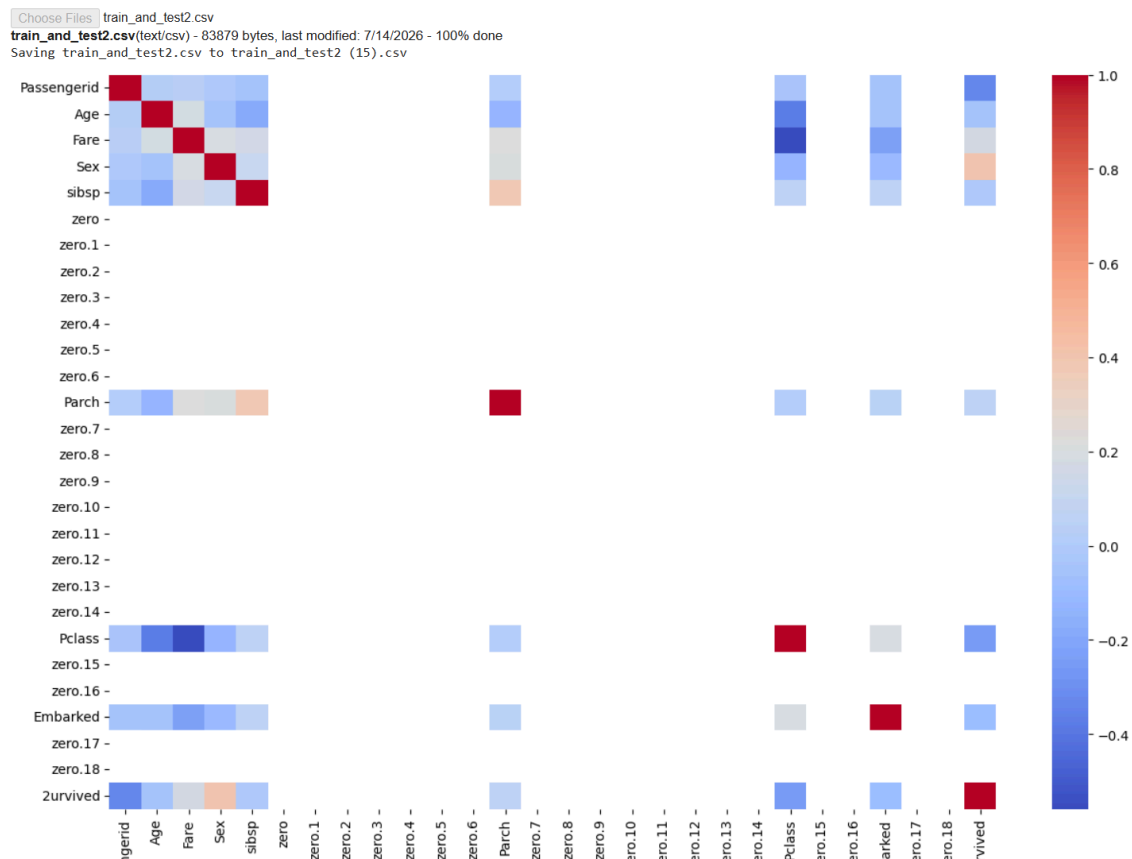
import matplotlib.pyplot as plt

plt.figure(figsize=(15,10))

sns.heatmap(df.corr(), annot=False, cmap="coolwarm")

plt.show()
```

## OUTPUT



## Explanation

A correlation matrix is generated to understand relationships between variables. A heatmap is used for better visualization.

## Q12

### #12 Feature Engineering

```
numeric = df.select_dtypes(include="number").columns

df["New_Feature"] = df[numeric[0]] + df[numeric[1]]

print(df.head())
```

## OUTPUT

train\_and\_test2.csv

**train\_and\_test2.csv**(text/csv) - 83879 bytes, last modified: 7/14/2026 - 100% done

Saving train\_and\_test2.csv to train\_and\_test2 (16).csv

|   | Passengerid | Age  | Fare    | Sex | sibsp | zero | zero.1 | zero.2 | zero.3 | \ |
|---|-------------|------|---------|-----|-------|------|--------|--------|--------|---|
| 0 | 1           | 22.0 | 7.2500  | 0   | 1     | 0    | 0      | 0      | 0      |   |
| 1 | 2           | 38.0 | 71.2833 | 1   | 1     | 0    | 0      | 0      | 0      |   |
| 2 | 3           | 26.0 | 7.9250  | 1   | 0     | 0    | 0      | 0      | 0      |   |
| 3 | 4           | 35.0 | 53.1000 | 1   | 1     | 0    | 0      | 0      | 0      |   |
| 4 | 5           | 35.0 | 8.0500  | 0   | 0     | 0    | 0      | 0      | 0      |   |

|   | zero.4 | ... | zero.13 | zero.14 | Pclass | zero.15 | zero.16 | Embarked | zero.17 | \ |
|---|--------|-----|---------|---------|--------|---------|---------|----------|---------|---|
| 0 | 0      | ... | 0       | 0       | 3      | 0       | 0       | 2.0      | 0       |   |
| 1 | 0      | ... | 0       | 0       | 1      | 0       | 0       | 0.0      | 0       |   |
| 2 | 0      | ... | 0       | 0       | 3      | 0       | 0       | 2.0      | 0       |   |
| 3 | 0      | ... | 0       | 0       | 1      | 0       | 0       | 2.0      | 0       |   |
| 4 | 0      | ... | 0       | 0       | 3      | 0       | 0       | 2.0      | 0       |   |

|   | zero.18 | Survived | New_Feature |
|---|---------|----------|-------------|
| 0 | 0       | 0        | 23.0        |
| 1 | 0       | 1        | 40.0        |
| 2 | 0       | 1        | 29.0        |
| 3 | 0       | 1        | 39.0        |
| 4 | 0       | 0        | 40.0        |

[5 rows x 29 columns]

## Explanation

A new feature is created by combining existing features to improve model performance.

### **Q13**

#13 Convert Data Types

```
print(df.dtypes)

df["Age"] = df["Age"].astype(float)

print(df.dtypes)
```

### **OUTPUT**

PassengerId int64

Age float64

Fare float64

Sex int64

sibsp int64

zero int64

zero.1 int64

zero.2 int64

zero.3 int64

zero.4 int64

zero.5 int64

zero.6 int64

Parch int64

zero.7 int64

zero.8 int64

zero.9 int64

zero.10 int64

zero.11 int64

zero.12 int64

zero.13 int64

zero.14 int64

Pclass int64

zero.15 int64

zero.16 int64

Embarked float64

zero.17 int64

zero.18 int64

Survived int64

dtype: object

PassengerId int64

Age float64

Fare float64

Sex int64

sibsp int64

zero int64

zero.1 int64

zero.2 int64

zero.3 int64

zero.4 int64

zero.5 int64

zero.6 int64

Parch int64

zero.7 int64

zero.8 int64

zero.9 int64

zero.10 int64

zero.11 int64

zero.12 int64

zero.13 int64

zero.14 int64

## Explanation

Data types are converted wherever necessary to ensure consistency and compatibility.

### Q14

#14 Handle inconsistent/noisy data

```
df.columns = df.columns.str.strip()

df.replace("?", pd.NA, inplace=True)

df.replace("NA", pd.NA, inplace=True)

print(df.head())
```

## OUTPUT

train\_and\_test2.csv

**train\_and\_test2.csv**(text/csv) - 83879 bytes, last modified: 7/14/2026 - 100% done

Saving train\_and\_test2.csv to train\_and\_test2 (18).csv

|   | Passengerid | Age  | Fare    | Sex | sibsp | zero | zero.1 | zero.2 | zero.3 | \ |
|---|-------------|------|---------|-----|-------|------|--------|--------|--------|---|
| 0 | 1           | 22.0 | 7.2500  | 0   | 1     | 0    | 0      | 0      | 0      |   |
| 1 | 2           | 38.0 | 71.2833 | 1   | 1     | 0    | 0      | 0      | 0      |   |
| 2 | 3           | 26.0 | 7.9250  | 1   | 0     | 0    | 0      | 0      | 0      |   |
| 3 | 4           | 35.0 | 53.1000 | 1   | 1     | 0    | 0      | 0      | 0      |   |
| 4 | 5           | 35.0 | 8.0500  | 0   | 0     | 0    | 0      | 0      | 0      |   |

|   | zero.4 | ... | zero.12 | zero.13 | zero.14 | Pclass | zero.15 | zero.16 | Embarked | \ |
|---|--------|-----|---------|---------|---------|--------|---------|---------|----------|---|
| 0 | 0      | ... | 0       | 0       | 0       | 3      | 0       | 0       | 2.0      |   |
| 1 | 0      | ... | 0       | 0       | 0       | 1      | 0       | 0       | 0.0      |   |
| 2 | 0      | ... | 0       | 0       | 0       | 3      | 0       | 0       | 2.0      |   |
| 3 | 0      | ... | 0       | 0       | 0       | 1      | 0       | 0       | 2.0      |   |
| 4 | 0      | ... | 0       | 0       | 0       | 3      | 0       | 0       | 2.0      |   |

|   | zero.17 | zero.18 | Survived |
|---|---------|---------|----------|
| 0 | 0       | 0       | 0        |
| 1 | 0       | 0       | 1        |
| 2 | 0       | 0       | 1        |
| 3 | 0       | 0       | 1        |
| 4 | 0       | 0       | 0        |

[5 rows x 28 columns]

## **Explanation**

Noisy and inconsistent data such as extra spaces, unknown values, and invalid entries are cleaned.

## **Q15**

#15 Train-Test Split

```
from sklearn.model_selection import train_test_split

df.rename(columns={"Survived": "Survived"}, inplace=True)

X = df.drop("Survived", axis=1)

y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.2,

    random_state=42

)

print(X_train.shape)

print(X_test.shape)
```

## **OUTPUT**

Choose Files train\_and\_test2.csv

train\_and\_test2.csv(text/csv) - 83879 bytes, last modified: 7/14/2026 - 100% d  
Saving train\_and\_test2.csv to train\_and\_test2 (23).csv  
(1047, 27)  
(262, 27)

## **Explanation**

The dataset is divided into training and testing sets using Scikit-learn. The training set is used to build the model, while the testing set evaluates its performance.

### **Q16**

#16 Data Balancing using SMOTE

```
from sklearn.impute import SimpleImputer
```

```
from imblearn.over_sampling import SMOTE
```

```
# Remove unwanted 'zero' columns
```

```
df = df.loc[:, ~df.columns.str.startswith("zero")]
```

```
# Rename target column
```

```
df.rename(columns={"2urvived": "Survived"}, inplace=True)
```

```
# Separate features and target
```

```
X = df.drop("Survived", axis=1)
```

```
y = df["Survived"]
```

```
# Encode categorical columns

label_encoder = LabelEncoder()

for col in X.select_dtypes(include="object").columns:

    X[col] = label_encoder.fit_transform(X[col].astype(str))

# Fill missing values

imputer = SimpleImputer(strategy="median")

X = pd.DataFrame(imputer.fit_transform(X), columns=X.columns)

# Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.2,

    random_state=42,

    stratify=y

)

# Apply SMOTE

smote = SMOTE(random_state=42)

X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)


print("Before SMOTE:")

print(y_train.value_counts())

print("\nAfter SMOTE:")

print(y_train_sm.value_counts())
```

## OUTPUT

```
Choose Files train_and_test2.csv
train_and_test2.csv(text/csv) - 83879 bytes, last modified: 7/14/2026 - 100% d
Saving train_and_test2.csv to train_and_test2 (26).csv
Before SMOTE:
Survived
0      773
1      274
Name: count, dtype: int64

After SMOTE:
Survived
0      773
1      773
Name: count, dtype: int64
```

## Explanation

SMOTE (Synthetic Minority Over-sampling Technique) is applied to balance the target classes and improve model performance.

## Q17

```
#17 Distribution before and after preprocessing

#before

df.hist(figsize=(15,10))

plt.show()

#after

for col in df.select_dtypes(include="number").columns:
```

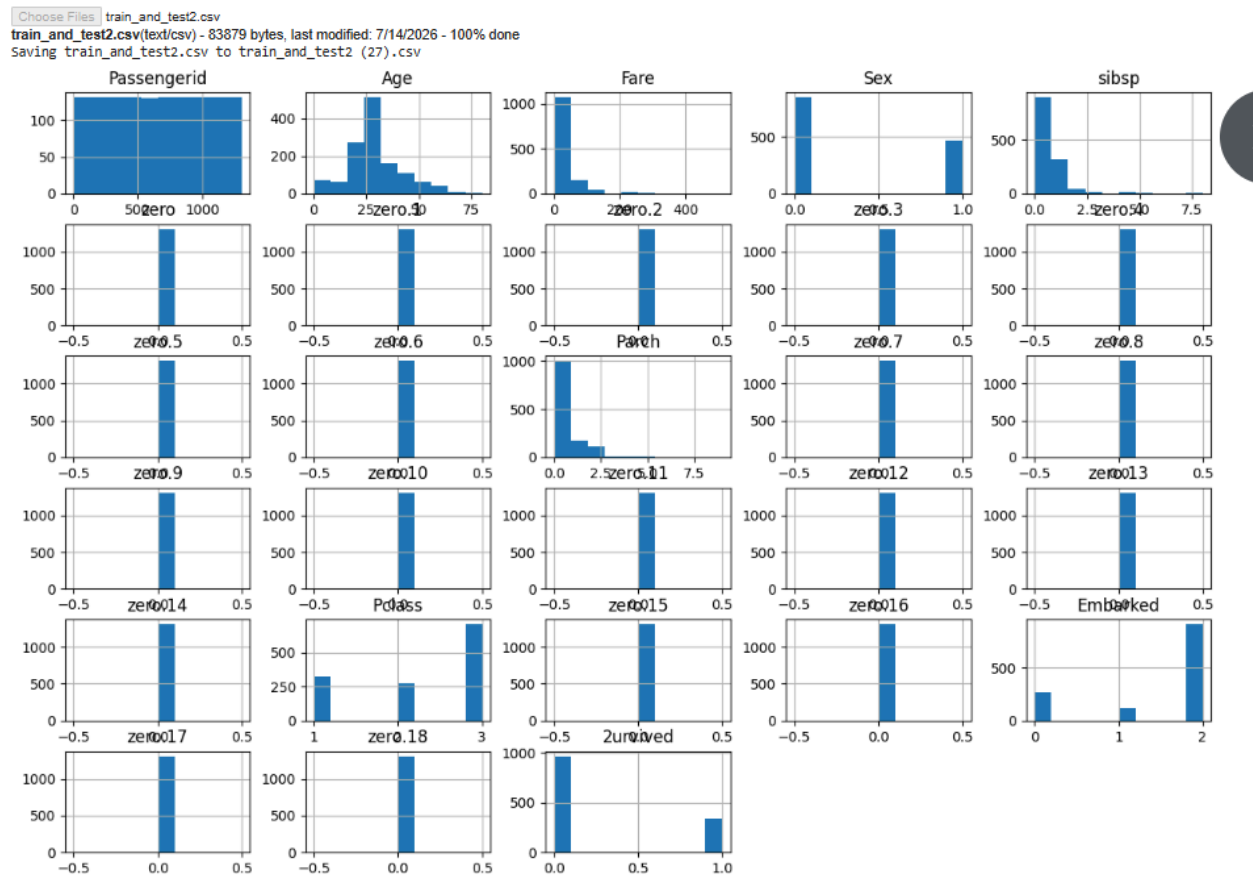
```
plt.figure(figsize=(6,3))

plt.hist(df[col])

plt.title(col)

plt.show()
```

## OUTPUT



## Explanation

Histograms are plotted before and after preprocessing to observe the changes in data distribution.

### **Q18**

#18 Compare before and after preprocessing

```
print("Shape:", df.shape)
```

```
print(df.head())
```

```
print(df.describe())
```

### **OUTPUT**

|   |   |     |   |   |   |   |   |   |     |
|---|---|-----|---|---|---|---|---|---|-----|
| - | - | ... | - | - | - | - | - | - | ... |
| 1 | 0 | ... | 0 | 0 | 0 | 1 | 0 | 0 | 0.0 |
| 2 | 0 | ... | 0 | 0 | 0 | 3 | 0 | 0 | 2.0 |
| 3 | 0 | ... | 0 | 0 | 0 | 1 | 0 | 0 | 2.0 |
| 4 | 0 | ... | 0 | 0 | 0 | 3 | 0 | 0 | 2.0 |

|   |         |         |          |
|---|---------|---------|----------|
|   | zero.17 | zero.18 | Survived |
| 0 | 0       | 0       | 0        |
| 1 | 0       | 0       | 1        |
| 2 | 0       | 0       | 1        |
| 3 | 0       | 0       | 1        |
| 4 | 0       | 0       | 0        |

[5 rows x 28 columns]

|       |             |             |             |             |             |
|-------|-------------|-------------|-------------|-------------|-------------|
|       | Passengerid | Age         | Fare        | Sex         | sibsp \     |
| count | 1309.000000 | 1309.000000 | 1309.000000 | 1309.000000 | 1309.000000 |
| mean  | 655.000000  | 29.503186   | 33.281086   | 0.355997    | 0.498854    |
| std   | 378.020061  | 12.905241   | 51.741500   | 0.478997    | 1.041658    |
| min   | 1.000000    | 0.170000    | 0.000000    | 0.000000    | 0.000000    |
| 25%   | 328.000000  | 22.000000   | 7.895800    | 0.000000    | 0.000000    |
| 50%   | 655.000000  | 28.000000   | 14.454200   | 0.000000    | 0.000000    |
| 75%   | 982.000000  | 35.000000   | 31.275000   | 1.000000    | 1.000000    |
| max   | 1309.000000 | 80.000000   | 512.329200  | 1.000000    | 8.000000    |

|       |        |        |        |        |        |     |         |         |           |
|-------|--------|--------|--------|--------|--------|-----|---------|---------|-----------|
|       | zero   | zero.1 | zero.2 | zero.3 | zero.4 | ... | zero.12 | zero.13 | zero.14 \ |
| count | 1309.0 | 1309.0 | 1309.0 | 1309.0 | 1309.0 | ... | 1309.0  | 1309.0  | 1309.0    |
| mean  | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0       |
| std   | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0       |
| min   | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0       |
| 25%   | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0       |
| 50%   | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0       |
| 75%   | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0       |
| max   | 0.0    | 0.0    | 0.0    | 0.0    | 0.0    | ... | 0.0     | 0.0     | 0.0       |

|       |             |         |         |             |         |           |
|-------|-------------|---------|---------|-------------|---------|-----------|
|       | Pclass      | zero.15 | zero.16 | Embarked    | zero.17 | zero.18 \ |
| count | 1309.000000 | 1309.0  | 1309.0  | 1307.000000 | 1309.0  | 1309.0    |
| mean  | 2.294882    | 0.0     | 0.0     | 1.492731    | 0.0     | 0.0       |
| std   | 0.837836    | 0.0     | 0.0     | 0.814626    | 0.0     | 0.0       |
| min   | 1.000000    | 0.0     | 0.0     | 0.000000    | 0.0     | 0.0       |
| 25%   | 2.000000    | 0.0     | 0.0     | 1.000000    | 0.0     | 0.0       |
| 50%   | 3.000000    | 0.0     | 0.0     | 2.000000    | 0.0     | 0.0       |
| 75%   | 3.000000    | 0.0     | 0.0     | 2.000000    | 0.0     | 0.0       |
| max   | 3.000000    | 0.0     | 0.0     | 2.000000    | 0.0     | 0.0       |

|       |             |
|-------|-------------|
|       | Survived    |
| count | 1309.000000 |
| mean  | 0.261268    |
| std   | 0.439494    |
| min   | 0.000000    |
| 25%   | 0.000000    |
| 50%   | 0.000000    |
| 75%   | 1.000000    |
| max   | 1.000000    |

## **Explanation**

The original dataset is compared with the cleaned dataset to observe improvements such as:

- Missing values handled
- Duplicates removed
- Outliers removed
- Encoded categorical variables
- Scaled numerical features

## **Q19**

```
#19 Save cleaned dataset
```

```
df.to_csv("cleaned_dataset.csv", index=False)
```

```
print("Dataset Saved Successfully")
```

## **OUTPUT**

train\_and\_test2.csv

**train\_and\_test2.csv**(text/csv) - 83879 bytes, last modified: 7/14/2026 - 100% done

Saving train\_and\_test2.csv to train\_and\_test2 (29).csv

Dataset Saved Successfully

## **Explanation**

The cleaned and preprocessed dataset is saved as a new CSV file for future use.